Hands-on Activity 9.2 Customized Visualizations using Seaborn

Instructions:

Create a Python notebook to answer all shown procedures, exercises and analysis in this section.

Resources:

Download the following datasets: fb_stock_prices_2018.csv, Download earthquakes-1.csv

Procedures:

- 9.4 Introduction to Seaborn
- 9.5 Formatting Plots
- 9.6 Customizing Visualizations

Data Analysis:

- **9.4 Introduction to Seaborn:**

I learned that Seaborn is built on top of Matplotlib and it makes data visualization a lot easier and cleaner. It's designed to work well with DataFrames, and the default styles already look professional. Using functions like sns.boxplot() and sns.histplot(), I can create good-looking plots with just a few lines.

- **9.5 Formatting Plots:**

This part was all about improving the appearance of my graphs. I learned how to add proper titles, labels, and adjust things like rotation on tick labels to make the plot more readable. Even small formatting changes made a big difference in making the plots easier to understand.

- **9.6 Customizing Visualizations:**

Here I got to explore things like choosing color palettes, adjusting transparency, and turning on grid lines. It helped me match the visual style of the plot to the type of data I was showing. Customizing the visuals made it more engaging and clearer for comparisons.

Supplementary Activity

Using the CSV files provided and what we have learned so far in this module complete the following exercises:

1. Using seaborn, create a heatmap to visualize the correlation coefficients between earthquake magnitude and whether there was a tsunami with the magType of mb.
2. Create a box plot of Facebook volume traded and closing prices, and draw reference lines for the bounds of a Tukey fence with a multiplier of 1.5. The bounds will be at Q1 - 1.5 * IQR and Q3 + 1.5 * IQR. Be sure to use the quantile() method on the data to make this

easier. (Pick whichever orientation you prefer for the plot, but make sure to use subplots.)

3. Fill in the area between the bounds in the plot from exercise #2.
4. Use axvspan() to shade a rectangle from '2018-07-25' to '2018-07-31', which marks the large decline in Facebook price on a line plot of the closing price.
5. Using the Facebook stock price data, annotate the following three events on a line plot of the closing price:
   - Disappointing user growth announced after close on July 25, 2018
   - Cambridge Analytica story breaks on March 19, 2018 (when it affected the market)
   - FTC launches investigation on March 20, 2018
6. Modify the reg_resid_plots() function to use a matplotlib colormap instead of cycling between two colors. Remember, for this use case, we should pick a qualitative colormap or make our own.

Summary/Conclusion:

Provide a summary of your learnings and the conclusion for this activity.

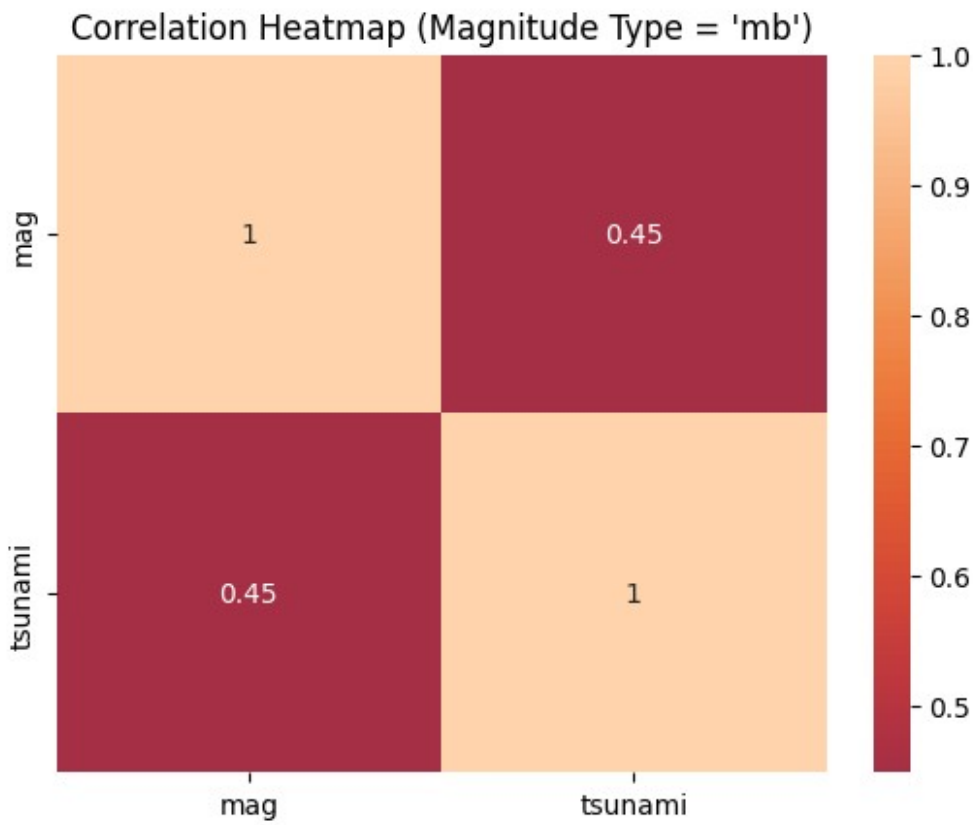
I learned how to use different types of plots to better understand data, like line plots, histograms, KDEs, and box plots. I also practiced using rolling and resample for time series data, and how to customize visuals with Matplotlib and Seaborn. It made it easier to see patterns and differences in the data. I might not remember all the details right away, but I have a better idea of how to start and what tools to use when analyzing data.

```
import numpy as np
import pandas as pd
from google.colab import drive
drive.mount('/content/drive')
```

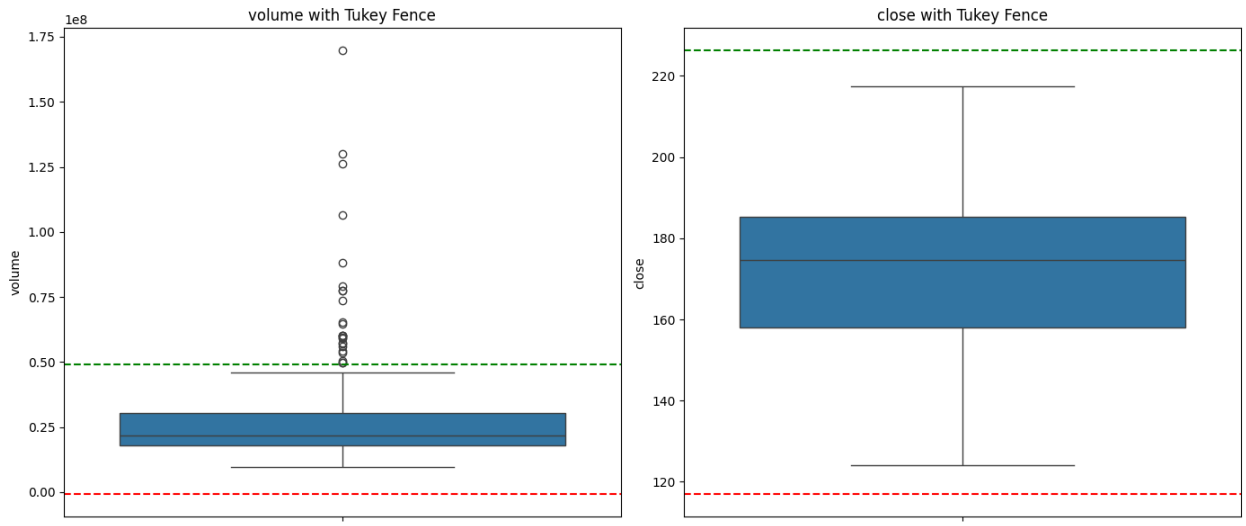
```
Mounted at /content/drive
```

```
%matplotlib inline
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sns
import pandas as pd

fb =
pd.read_csv('/content/drive/MyDrive/Foldah/fb_stock_prices_2018.csv',
index_col='date', parse_dates=True)
quakes = pd.read_csv('/content/drive/MyDrive/Foldah/earthquakes-
1.csv')

mb_filter = quakes.query(" magType == 'mb'")
correlations = mb_filter[['mag','tsunami']].corr()
sns.heatmap(correlations, annot = True, center = 0)
plt.title("Correlation Heatmap (Magnitude Type = 'mb')")
```

```
Text(0.5, 1.0, "Correlation Heatmap (Magnitude Type = 'mb')")
```

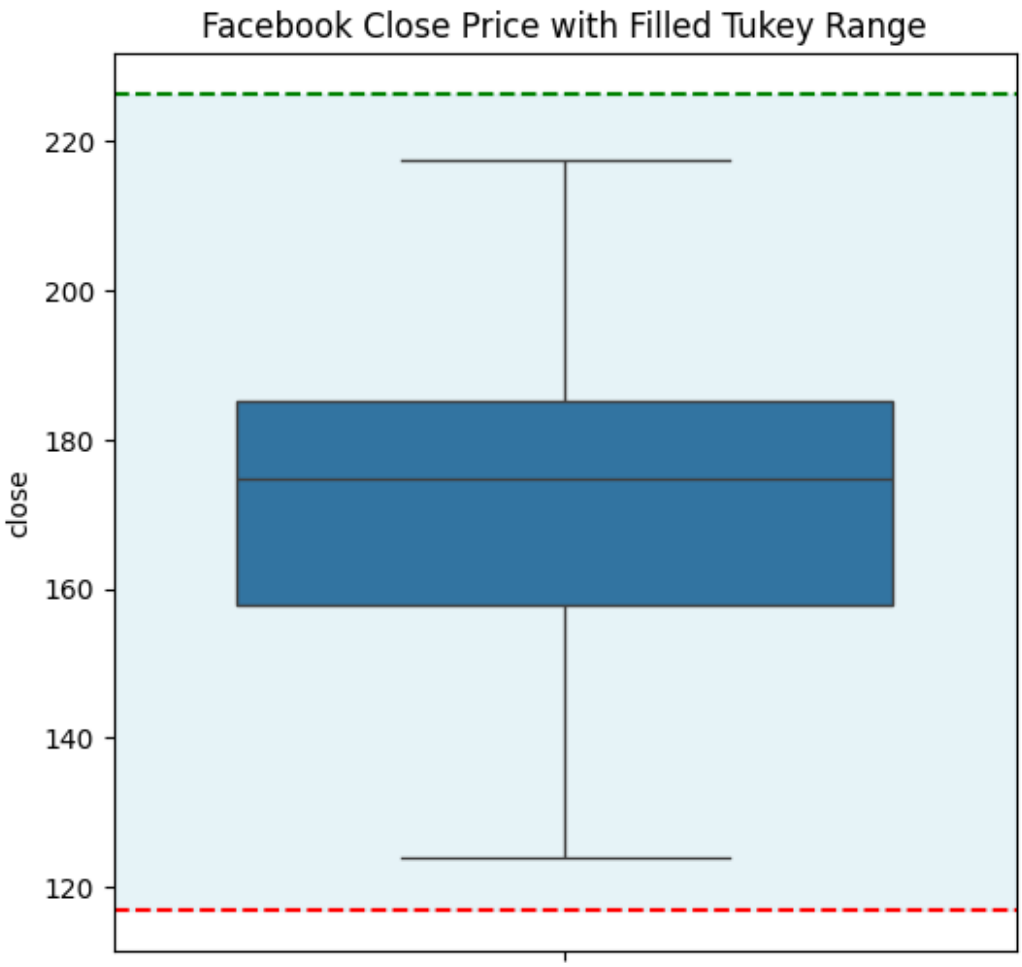



```
fig, axes = plt.subplots(1, 2, figsize=(14, 6))

for i, col in enumerate(['volume', 'close']):
    q1 = fb[col].quantile(0.25)
    q3 = fb[col].quantile(0.75)
    iqr = q3 - q1
    lower = q1 - 1.5 * iqr
    upper = q3 + 1.5 * iqr

    sns.boxplot(y=fb[col], ax=axes[i])
    axes[i].axhline(lower, color='red', linestyle='--')
    axes[i].axhline(upper, color='green', linestyle='--')
    axes[i].set_title(f'{col} with Tukey Fence')

plt.tight_layout()
```



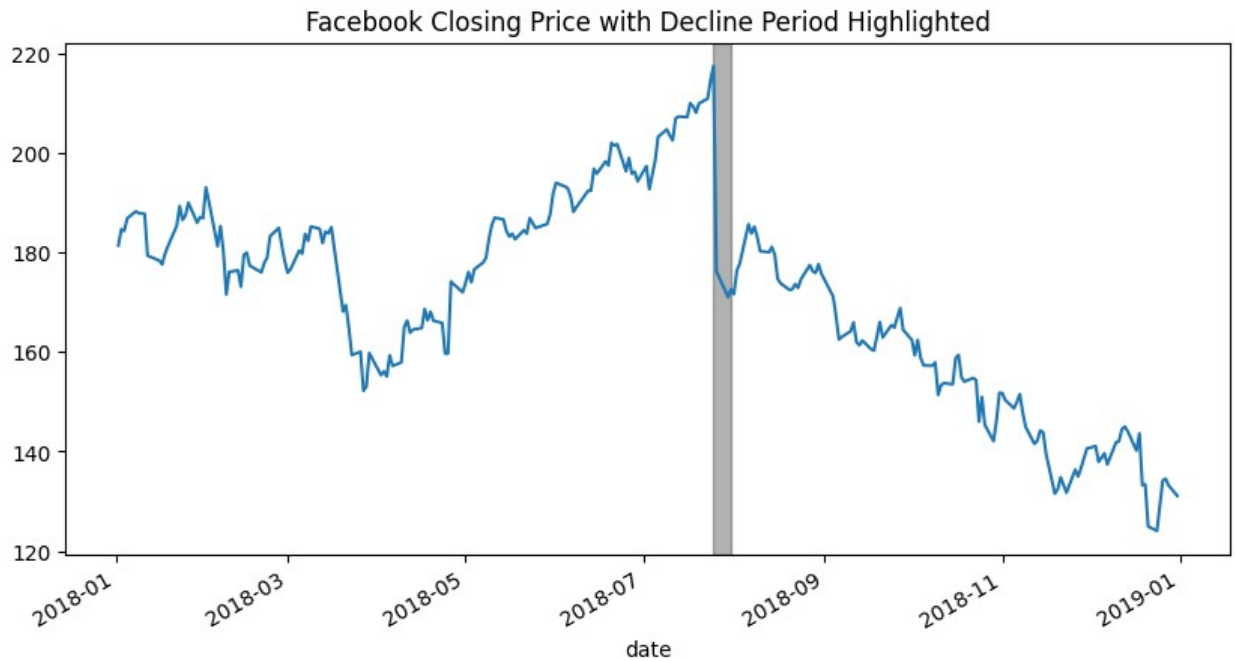

```
fig, ax = plt.subplots(figsize=(6, 6))

col = 'close'
q1 = fb[col].quantile(0.25)
q3 = fb[col].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr

sns.boxplot(y=fb[col], ax=ax)
ax.axhline(lower, color='red', linestyle='--')
ax.axhline(upper, color='green', linestyle='--')
ax.fill_betweenx(y=[lower, upper], x1=0, x2=1, color='lightblue',
alpha=0.3, transform=ax.get_yaxis_transform())

ax.set_title('Facebook Close Price with Filled Tukey Range')
```

```
Text(0.5, 1.0, 'Facebook Close Price with Filled Tukey Range')
```

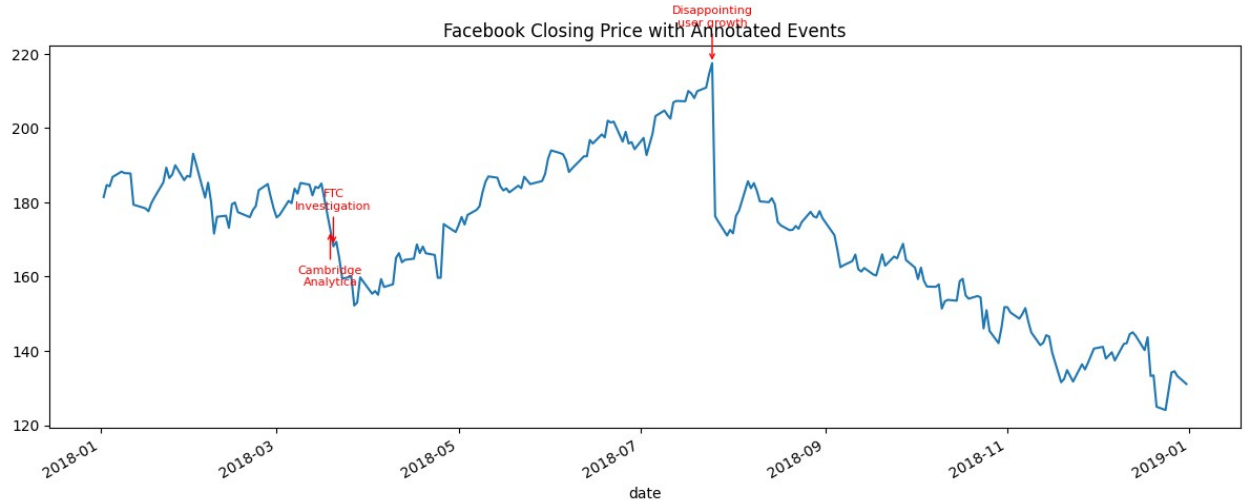



```
fig, ax = plt.subplots(figsize=(10, 5))

fb['close'].plot(ax=ax)
ax.axvspan(pd.to_datetime('2018-07-25'), pd.to_datetime('2018-07-31'),
color='black', alpha=0.3)
ax.set_title("Facebook Closing Price with Decline Period Highlighted")
```

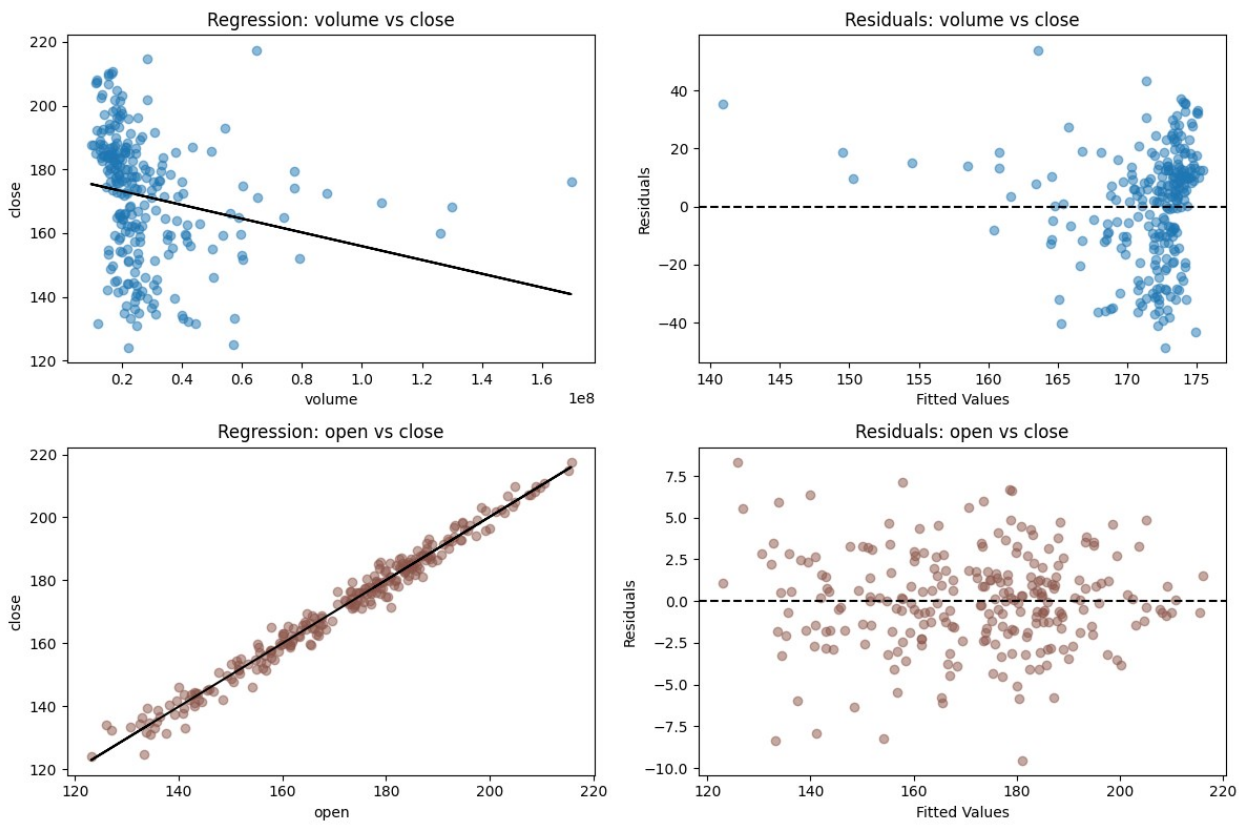
```
Text(0.5, 1.0, 'Facebook Closing Price with Decline Period
Highlighted')
```




```
fig, ax = plt.subplots(figsize=(12, 5))

fb['close'].plot(ax=ax)

events = [
    ('2018-07-25', 'Disappointing\nuser growth', 'above'),
    ('2018-03-19', 'Cambridge\nAnalytica', 'below'),
    ('2018-03-20', 'FTC\nInvestigation', 'above'),
]

for date_str, label, position in events:
    date = pd.to_datetime(date_str)
    price = fb.loc[date, 'close']
    y_offset = 10 if position == 'above' else -15
    ax.annotate(
        label,
        xy=(date, price),
        xytext=(date, price + y_offset),
        arrowprops=dict(arrowstyle='->', color='red'),
        fontsize=8,
        ha='center',
        color='red'
    )

ax.set_title('Facebook Closing Price with Annotated Events')
plt.tight_layout()
```




```
import matplotlib as mpl
from sklearn.linear_model import LinearRegression

def reg_resid_plots(data):

    num_cols = data.select_dtypes(include='number').columns
    y_col = num_cols[-1]
    feature_cols = num_cols[:-1]

    cmap = mpl.colormaps.get_cmap('tab10')
    colors = [cmap(i / len(feature_cols)) for i in
range(len(feature_cols))]

    fig, axes = plt.subplots(len(feature_cols), 2, figsize=(12, 4 *
len(feature_cols)))

    if len(feature_cols) == 1:
        axes = [axes]

    for i, (col, color) in enumerate(zip(feature_cols, colors)):
        X = data[[col]].dropna()
        y = data.loc[X.index, y_col]

        model = LinearRegression().fit(X, y)
        predictions = model.predict(X)
        residuals = y - predictions

        axes[i][0].scatter(X, y, color=color, alpha=0.5)
        axes[i][0].plot(X, predictions, color='black')
        axes[i][0].set_xlabel(col)
        axes[i][0].set_ylabel(y_col)
        axes[i][0].set_title(f'Regression: {col} vs {y_col}')

        axes[i][1].scatter(predictions, residuals, color=color,
```

```
alpha=0.5)
        axes[i][1].axhline(0, color='black', linestyle='--')
        axes[i][1].set_xlabel('Fitted Values')
        axes[i][1].set_ylabel('Residuals')
        axes[i][1].set_title(f'Residuals: {col} vs {y_col}')

    plt.tight_layout()
    return axes

reg_resid_plots(fb[['volume', 'open', 'close']])
```

```
array([[<Axes: title={'center': 'Regression: volume vs close'},
xlabel='volume', ylabel='close'>,
        <Axes: title={'center': 'Residuals: volume vs close'},
xlabel='Fitted Values', ylabel='Residuals'>],
       [<Axes: title={'center': 'Regression: open vs close'},
xlabel='open', ylabel='close'>,
        <Axes: title={'center': 'Residuals: open vs close'},
xlabel='Fitted Values', ylabel='Residuals'>]],
      dtype=object)
```